

Transactional recovery read — stall cheat sheet

KafkaPartitionPersistence · read_committed drain to the captured high-watermark · poll 10 ms · deadline 3 min · legit wait ~70 s (txn.timeout 60 s + abort scan 10 s) · worked example HW = 1000

CONTROL FLOW one loop, two kinds of turn, two exits

SETUP (once, before the loop)

B1 highWatermark = endOffsets(read_uncommitted) · the high watermark

B2 lastStableOffset = endOffsets(read_committed) · the LSO (reader's own end)

B3 if lastStableOffset < highWatermark → WARN "recovery waits for open transaction(s)"

B4 targetOffset = highWatermark · the target: captured once, fixed for the read (never the LSO)

LOOP (tailRecM · one turn ≈ 10 ms)

B5 offset = position()

if offset ≥ targetOffset → EXIT ✓ read complete

B6 advanced? (offset ≠ last.position)

YES → ADVANCING turn · reset clock (progressAt = now)

NO → WAITING turn · B7 if now - progressAt ≥ 3 min → EXIT ✗ B8 RecoveryReadStalledError

B9 logEnd < target ? truncation : outlived

else · every 5 s → INFO "no progress ..."

both turns end: poll(10.millis) ; fold batch → back to B5

TIMELINE — FOUR SCENARIOS SIDE BY SIDE a walk through the loop above

target = high watermark, captured once at B1/B4 and held constant · ♻ = the loop, one per ~10 ms turn · advancing turn (B6) poll → batch, position climbs · waiting turn (B7) poll → empty, position frozen, clock ticks

wall clock	D — no wait (takeover-abort)	A — awaiting open txn (resolves)	B — hanging transaction	C — truncation
0 entry	B1–B4: LSO 1000 = HW → no warn, target 1000 (own txn aborted at takeover)	B1–B4: LSO 950 < HW 1000 → WARN, target 1000	same: LSO 950 → WARN, target 1000	LSO 1000 = HW → no warn, target 1000
0–~50 ms climb	♻ advancing turns (B6) · position 0 → 1000 → B5 exit ✓ complete @1000 (only advancing turns — never B7)	♻ advancing turns (B6) · position 0 → 950, each poll a batch	♻ advancing turns (B6) · position 0 → 950	♻ advancing turns (B6) · climbing... ⚡ election → log end 900
first stall (no log yet)	— done —	waiting turn (B7) begins · stuck@950, stalledFor=0	waiting turn (B7) · stuck@950	waiting turn (B7) · stuck@900 (nothing past 900)
5 s → 70 s steady loop	— done —	♻ waiting turns (B7) ≈500×/5 s · poll(10ms) empty, position frozen, B8 false · every 5 s → INFO "... Ns"	same @950	same @900
≈70 s	— done —	⚡ txn resolves → advancing turn: poll returns 950–999 → position 1000 → B5 exit ✓ complete @1000	still waiting @950 (♻ INFO every 5 s, B8 false)	still waiting @900 (♻ INFO every 5 s)
180 s deadline	— done —	— done —	waiting turn · stalledFor≥3min → B8 ✗ → B9 logEnd 1000 ≥ target → "outlived deadline" → RecoveryReadStalledError@950	→ B8 ✗ → B9 logEnd 900 < target → "truncation" → ...@900

WHAT YOU SEE IN THE LOGS verbatim; blue = interpolated values

at	level	message
B3 start	WARN	Snapshot topic <t> partition <p> recovery waits for open transaction(s): last-stable-offset <LSO> below target <HW>; normally resolves within the pinning producer's transaction.timeout.ms plus the broker's abort scan
B7 waiting	INFO	Snapshot topic read making no progress at offset <o>, target <t>, stalled for <N>s (repeats every 5 s)
B8 fail	ERROR	recovery read of <tp> made no progress at offset <pos>, short of target <target>, past the stall deadline - failing rather than hanging or silently recovering possibly incomplete state; <diagnosis>
B9 ↴ trunc.	diag	the log end regressed to <logEnd>, below the captured target: log truncation after an unclean leader election - acknowledged snapshot records are lost
B9 ↴ outlived	diag	the log end <logEnd> still covers the target: an open transaction outlived the deadline - ... detect and abort it with kafka-transactions.sh

Every scenario runs the same loop — each turn is either an advancing turn (B6: poll returned a batch, position climbs) or a waiting turn (B7: poll came back empty, position frozen, clock ticks). D is all advancing turns; A/B/C add a run of waiting turns. A ≡ B until 70 s — identical waiting-turn runs; the only difference is whether a poll returns records before the clock reaches B8. No separate "hang" detector — the deadline is it, discovered late on an ordinary turn, never a scheduled wake-up. B vs C differ only at B3 (warn fires / silent) and B9 (logEnd vs target). Reader is group-less, so max.poll.interval.ms eviction can't unwind the loop — only ✓ B5 (records) or ✗ B8 (deadline) exits it.